Kingdom of Saudi Arabia
Ministry of Education
Imam Abdulrahman bin Faisal University
College of Science and Humanities-Jubail
Computer science Department

Software Engineering
Term 1 – 2024/2025

Software Quality Assurance Test Plan

For

HealthyCooking System

Version 1.0

CS Year 4 - G5 S3

2nd of December 2025

Software Test Plan (STP)

This document is an annotated Software Test Plan outline adapted from the IEEE Standard for Software Test Documentation
(Std 829-1998).

TABLE OF CONTENTS

Table of Tables

1. Introduction

This section of the Software Testing Plan (STP) provides an overview detailing the HealthyCooking testing strategy and deliverables, including objectives, scope, reference materials, and definitions The HealthyCooking program introduction here provides a comprehensive overview of the project, including deliverables, benchmarks, tests, and abbreviations used. It also identifies the personnel responsible for testing, the necessary resources and schedule, and the associated risks that affect everyone involved in the plan.

1.1 Objectives

The testing plan aims to define our system and its features, identifying the elements and testing methods we use in the HealthyCooking application This includes the main interfaces, testing various options and required procedures, performance comparisons, and overall user experience. It also encompasses all the aforementioned reports, tests, and comparisons between different features and characterist

1.2 Testing Strategy

The strategy for testing the healthy cooking application relies on documenting the test plan as shown in this table, which summarizes the function and department.

Sections	Functional
Introduction	Explanation of the testing strategy we rely on
Test Items	This includes selecting and defining application testing elements.
Features to Be Tested	All the different ideas, collections, and features in the application that we will be testing
Features Not to Be Tested	All the ideas, collections, and features in the various applications will not be tested, with reasons explained below.
Approach	The methods used in our plan are described here, along with their application testing technique. **The approaches used in this plan are:** • **Component Testing** • **Integration Testing** • **Conversion Testing** • **Job Stream Testing** • **Interface Testing**

	• **Security Testing** • **Recovery Testing** • **Performance Testing** • **Regression Testing** • **Acceptance Testing** • **Beta Testing**
Pass/Fail Criteria	The standard used is whether he passed or failed.
Testing Process	The testing activities in this section define the methods and criteria used.
Environmental Requirements	The required and desired characteristics for the test environment will be defined in this section.
Change Management Procedures	The STP change management process will be defined
Plan Approvals	Team members' approvals of the designed testing plan

Table 3: Testing stratigies

1.3 Scope

The testing plan for the healthy cooking app will be implemented and updated through several tasks and multiple phases of the app's development lifecycle. Testing plans evolve alongside the app's development, and at each level of testing, the duration of this lifecycle is scheduled for updates, distributed across releases, and managed through change management.

1.4 Reference Material

[1] IEEE Standard for Software Project, The Institute of Electrical and Electronics Engineers, 1998.

[2] HealthyCooking Software Requirements Specification (SDS) Document, 2025.

1.5 Definitions and Acronyms

The HealthyCooking the Software Testing Plan (STP) document includes several acronyms and abbreviations, which are defined in this document. This ensures that all project participants
have a consistent understanding of the terminology used throughout the document.

Acronym / Term	Definition
IEEE	Institute of Electrical and Electronics Engineers: Developer of key software engineering standards.
SRS	Software Requirements Specification: a document that describes all functional and non-functional requirements of the system.
SDS	Software Design Specification: A document that describes the architecture and design of a software system.
DB	Database: Central storage of all recipe data, user information and nutrition data.
API	Application Programming Interface: Channel that enables communication between software components.
STP	Software Test Plan
AI	Artificial Intelligence: Algorithms that learn and generate intelligent recommendations.
UI/UX	User Interface and User Experience: Design principles ensuring clarity and usability.
API Testing	Testing the API functionality to ensure the system behaves as expected during interactions with other services.
V & V	Verification and Validation

Table 4: Definitions and Acronyms

2. TEST ITEMS

This section identifies all items of the HealthyCooking System that will be tested, including modules, interfaces, data handling, security controls, API interactions, and database operations.

All test items are referenced according to the following documents:

- Software Requirements Specification (SRS)
- Software Design Specification (SDS)
- User Guide (HealthyCooking User Manual)
- Installation Guide (System Setup and Deployment Guide)
- Operations Guide (Maintenance Manual)
- Performance Requirements (response time, availability, load handling)
- Verification and Validation Plan (V&V strategy)
- UI Prototypes / Screen Designs
- System Architecture Diagrams
- Database ERD and Data Design
- Functional Requirements & Use Cases

System components to be tested include:

- Recipe Management
- Weekly Meal Planner
- Nutrition Tracking
- User Profile Module
- Personalized Meal Recommendation (AI)
- Notifications and Reminders
- Authentication and Authorization
- Database and API interactions

2.1 Program Modules

This subsection outlines the developer-level testing for each module in the HealthyCooking System. Developer testing for all modules will generally include integration and component testing.

1. User Interface Module

- UI testing will verify navigation flow, screen transitions, input validation, and responsiveness.
- Use-case-based testing will be performed for:
    - Recipe search interface
    - Weekly planner interface
    - Nutrition dashboard
    - Profile management page
    - Notification and reminder interface

2. Recipe Management Module

- Validate recipe storage, retrieval, filtering, and categorization.
- Test CRUD operations for recipes in the database.

3. Weekly Meal Planner Module

- Ensure recipe selection, editing, removing meals, and scheduling works correctly.
- Validate weekly plan generation and user modifications.

4. Nutrition Tracking Module

- Verify calorie calculations, daily summary, and updates to progress logs.
- Ensure correct integration with stored nutrition values.

5. AI Module: Personalized Meal Recommendation

- Test the AI for prediction accuracy using stored user preferences, allergies, and goals.
- Validate recommendation refresh based on updated data.

6. User Profile Module

- Verify profile updates, dietary preferences, health goals, and authentication data.

7. Notification Module

- Test reminder scheduling, sending alerts, and verifying system logs.

8. Database Module

- Perform SQL tests on the recipe, user, nutrition and reminder tables.
- Validate relational integrity-foreign keys and constraints.

2.2 Job Control Procedures

The HealthyCooking System uses job scheduling for automated tasks and timed services such as:

- Daily nutrition summary generation
- Scheduled reminders for meals
- Automated backup jobs
- AI model update jobs

Testing will verify the following:

- All scheduled tasks execute at correct times
- Failure recovery mechanisms (e.g., retry tasks, backup triggers)

- No conflicting jobs during peak hours
- Logs are updated correctly after every job execution

2.3 User Procedures

Testing will ensure all user documentation is:

- Correct
- Complete
- Easy to understand

Documents tested:

- User Guide (recipe search, weekly planning, tracking nutrition)
- Help screens inside the app
- Step-by-step instructions for updating profile
- FAQ and troubleshooting section

Testing activities:

- Check that all steps in the guide match system behavior
- Verify all screenshots are updated
- Validate instructions for errors and missing information

2.4 Operator Procedures

This section verifies that HealthyCooking can be supported and maintained by the operations team (Admin + Support):

Tests will include:

- Running the system using the admin dashboard
- Backup and restore procedures
- Monitoring errors and reviewing logs
- Help Desk workflow (e.g., resetting user accounts, reviewing reports)
- Ensuring operator manuals are clear and accurate

3. FEATURES TO BE TESTED

Evaluations of the HealthyCooking app will take place for evaluating the evaluation of HealthyCooking helps identify and further develop capabilities. All of the major aspect of HealthyCooking application will be examined for functional requirements, non-functional requirements, are the correct functioning of HealthyCooking, and HealthyCooking has not been allowed to incorrectly interact. Common patterns, edge cases and real world will also be utilized to evaluate the features of HealthyCooking app to demonstrate how reliable, accurate, and stable the system operates.

User authentication will be a critical focus for testing as it will verify the validity of

both login and signup methods. User profile management will also provide evidence of the ability of users to view and modify their profile information, including confirming that any modifications are maintained and shown accurately within the system's records.

The testing scope will encompass the following key features:

- Managing and organizing recipes can be done with Recipe Management.
- You can create/Edit/Store the meal planner on a weekly basis.
- You can add your meals in and get the nutritional value calculated and a total ingested calories calculated daily.
- Reminder & Notification System: you can add/Edit/Delete/Start and Modify notifications.
- User Interface & Navigation: you have to ensure that the User Interface works by having all Forms/Screen/Button comply with usability guidelines and that all Navigation paths work correctly.
- Security/Data Protection: Enforce Authorization, Encryption, and Authentication.
- Checking reaction time, system stability, and behavior during disruptions or network failures is known as performance and recovery.

When taken as a whole, these tests guarantee that the HealthyCooking application provides a reliable, effective, and user-friendly experience that complies with all criteria.

4. FEATURES NOT TO BE TESTED

Every important feature of the HealthyCooking System has to be tested in this document, otherwise it is not necessary to do that. In the following table, a list of features not to be tested is shown with the reason why.

Feature Not Tested	Reason Why
Viewing AI Recommendations	The system only displays suggestions, no impact on system behavior.
Viewing Favorite Recipes	The system only displays the recipes without it affecting any core functions.
Viewing Profile Information	The system only displays the information without it affecting any core functions.
AI Model Training	The system uses previously built model. No internal training is performed in this version.
Accessibility Testing	The system does not include accessibility features as screen reader and large font size support in this version.
Full integration with the external nutrition APIs	The system in this version doesn’t have a test environment for the APIs.
Data migration from previous systems	The system does not need this kind of test because there is no old system to migrate data from.

Table 5: Features Not to Be Tested

5. APPROACH

5.1 Component Testing

Sign-up:

Case Number	*Test Objective*	*Input Data*	*Expected Result*
C1	*Verify successful account creation*	*Valid name, email, strong password*	*Account is created successfully*
C2	*Verify system response for existing email*	*Email already registered*	*"Email already exists" message appears*
C3	*Verify required fields validation*	*Empty field(s)*	*"All fields are required" message appears*

Table 6: Sign-up component testing

Add Recipe :

Case Number	*Test Objective*	*Input Data*	*Expected Result*
C1	*Verify adding a recipe successfully*	*Title + Ingredients*	*Recipe is saved*
C2	*Verify missing title validation*	*Empty title*	*"Recipe title is required" message appears*
C3	*Verify unsupported image upload*	*file.exe*	*"Invalid file type" message appears*

Table 7: Add recipe component testing

Profile Management:

Case Number	*Test Objective*	*Input Data*	*Expected Result*
C1	*Verify profile name update*	*New name*	*Name updated successfully*
C2	*Verify no-change detection*	*No edits*	*"No changes detected" message displays*

C3	*Verify allergy/preferences update*	*Excluded ingredients*	*Preferences updated successfully*

Table 8: Profile management component testing

Main Home Page:

Case Number	Test Objective	Input Data	Expected Result
C1	*Verify recipe list display*	*Open app*	*Home page shows recipe list*
C2	*Verify navigation to recipe details*	*Tap a recipe card*	*Recipe details page opens*
C3	*Verify page refresh*	*Swipe down*	*Data refreshes successfully*

Table 9: Main homepage component testing

Delete Recipe:

Case Number	Test Objective	Input Data	Expected Result
C1	*Verify delete confirmation appears*	*Tap delete button*	*Confirmation dialog appears*
C2	*Verify successful deletion*	*Confirm delete*	*Recipe is removed*
C3	*Verify cancellation of delete action*	*Cancel delete*	*Recipe remains unchanged*

Table 10: Delete recipe component testing

Search Recipe:

Case Number	Test Objective	Input Data	Expected Result
C1	*Verify successful search*	*"Pasta"*	*Matching recipes displayed*
C2	*Verify empty search handling*	*""*	*"Enter a keyword" message appears*

C3	*Verify empty search handling*	*"XYZ123"*	*"No recipes found" message appears*

Table 11: Search recipe component testing

Track Calories:

Case Number	Test Objective	Input Data	Expected Result
C1	*Verify adding daily calories*	*450 kcal*	*Calories added to log*
C2	*Verify invalid input rejection*	*"abc"*	*Error message displayed*
C3	*Verify calorie item removal*	*Remove item*	*Item removed successfully*

Table 12: Track calories component testing

Meal Reminders:

Case Number	Test Objective	Input Data	Expected Result
C1	*Verify setting a reminder*	*Set time*	*Reminder saved successfully*
C2	*Verify invalid time handling*	*Invalid time*	*Error message appears*
C3	*Verify disabling a reminder*	*Toggle off*	*Reminder disabled*

Table 13: Meal remianders component testing

Log-in:

Case Number	Test Objective	Input Data	Expected Result
C1	*Verify successful login*	*Valid email + password*	*User logged in*
C2	*Verify incorrect password handling*	*Wrong password*	*"Incorrect password"*

			message appears
C3	*Verify invalid email format*	*invalidemail@*	*"Invalid email format" message appears*

Table 14: Login component testing

Edit Recipe:

Case Number	***Test Objective***	***Input Data***	***Expected Result***
C1	*Verify editing recipe title*	*New title*	*Title updated successfully*
C2	*Verify title required validation*	*Empty title*	*Error message appears*
C3	*Verify ingredient removal*	*Remove ingredient*	*Ingredient removed*

Table 15: Edit recipe component testing

Plan Weekly Meals:

Case Number	***Test Objective***	***Input Data***	***Expected Result***
C1	*Verify adding a meal to a day*	*Monday + Lunch + Pasta*	*Meal saved in plan*
C2	*Verify recipe required validation*	*No recipe selected*	*"Select a recipe" message appears*
C3	*Verify editing weekly plan*	*Change meal*	*Plan updated successfully*

Table 16: Plan weekly meals

Personalized Recommendations:

Case Number	***Test Objective***	***Input Data***	***Expected Result***
C1	*Verify recommendation display*	*User preferences*	*Valid recommendations shown*

C2	*Verify refresh functionality*	*Tap refresh*	*New recommendations displayed*
C3	*Verify saving a recommended recipe*	*Tap save*	*Recipe added to saved list*

Table 17: Personalized recommendations component testing

5.2 Integration Testing

5.2.1 User Authentication Component

Test ID	Auth_Integration_001
Prerequisite	User has the app installed and internet connection is available.
Test Procedure	• Access "Sign Up" and complete registration. • Log out and access "Log In" using the same credentials. • Perform "Reset Password" and verify email code. • Log in again using the new password.
Expected Result	All authentication functions interact correctly: new user is saved, login validation credentials, password reset updates the stored password, and system returns correct success/error messages
Actual Result	Same as expected result.
Verified (Yes/No)	Yes

Table 18: User authentication integration testing

5.2.2 User Profile Component

Test ID	Profile_Integration_002
Prerequisite	User must be logged in and have an existing profile
Test Procedure	• Open " My Profile". • Update profile details (name, dietary, preferences, calorie goal). • Save the changes. Reopen "My Profile" to confirm updates.
Expected Result	Updated information is stored correctly in the database, retrieved accurately, and displayed without errors.
Actual Result	Same as expected results.
Verified (Yes/No)	Yes

Table 19: User profile integration testing

5.2.3 Recipe Management Component

Test ID	Recipe_Integration_003
Prerequisite	User must be logged in. Recipes exist in the database.

Test Procedure	• Access “Add Recipe” and save a new recipe. • Access “Update Recipe” and modify details. • Access “Delete Recipe” and confirm removal. • Try to view the deleted recipe.
Expected Result	Recipes are inserted, updated, retrieved, and deleted successfully. Deleted recipe should not be accessible.
Actual Result	Same as expected results.
Verified (Yes/No)	Yes

Table 20: Recipe management integration testing

5.2.4 Weekly Meal Planner Component

Test ID	MealPlan_Integration_004
Prerequisite	User must have saved preferences + recipes available.
Test Procedure	• Open “Meal Planner”. • Request a weekly meal plan. • Review generated meals. • Save the weekly plan.
Expected Result	Meal planner retrieves preferences + recipes → system generates a valid meal plan → user can save and view it later.
Actual Result	Same as expected results.
Verified (Yes/No)	Yes

Table 21: Weekly meal planner integration testing

5.2.5 Calorie Tracker Component

Test ID	CalorieTracker_Integration_005
Prerequisite	User has a daily calorie goal set.
Test Procedure	• Log a meal with calories. • Open calorie summary page. • Add another meal to exceed daily goal.
Expected Result	System updates daily calories, displays correct total, and shows warning message when exceeding the goal.
Actual Result	Same as expected results.
Verified (Yes/No)	Yes

Table 22: Calorie tracker integration testing

5.2.6 Notification Component

Test ID	Notification_Integration_006
Prerequisite	User allows notification permissions.
Test Procedure	• Trigger a calorie alert by excluding daily limit. • Trigger a meal reminder.

Expected Result	Notifications are generated and delivered to the user's device successfully.
Actual Result	Same as expected results.
Verified (Yes/No)	Yes

Table 23: Notifications integration testing

5.3 Conversion Testing

Conversion testing focuses on verifying that any existing or legacy data can be transferred to the new HealthyCooking system without loss, corruption, or inconsistency. The main objective of this testing is to ensure that user profiles, saved recipes, and nutritional information remain accurate and usable after conversion to the new data format. It also checks that invalid or corrupted data is detected and handled properly during the conversion process.

Case Number	*Test Objective*	*Input Data*	*Expected Result*
C1	Verify successful conversion of user profiles	Existing user profile data in old format	User profile is converted correctly and all fields appear in the new system
C2	Verify conversion of recipe nutritional data	Old-format recipe nutrition values	Nutritional information is converted and displayed with the same values in the new system
C3	Verify conversion of saved recipes list	List of saved recipes in old format	All saved recipes appear correctly in the new system with the same count
C4	Verify handling of corrupted data during conversion	Corrupted or incomplete data file	System displays an error message and does not import corrupted data
C5	Verify that duplicate records are not created after conversion	Data file with duplicate recipe entries	System avoids creating duplicate records and keeps only one instance of each recipe

Table 24: Conversation testing

5.4 Job Stream Testing

Job Stream Testing focuses on verifying that the HealthyCooking application can operate smoothly in a production-like environment. Even though the system is not fully implemented, AI-based testing tools were used to simulate the sequence of tasks that the application performs during normal use.

The test checks that core operations such as launching the app, logging in, retrieving recipes, generating AI suggestions, saving user preferences, and accessing meal plans run in the correct order without interruption. The purpose of this testing is to ensure that every job in the workflow responds properly, completes successfully, and does not impact the following tasks.

This testing helps confirm that the application can be executed, supported, and maintained when deployed in a real operational environment.

5.5 Interface Testing

Interface testing involves assessing all interfaces of the HealthyCooking application to verify that screens, functional buttons, navigation elements, and data transfer components function properly and convey information correctly. This guarantees that users can effortlessly engage with functionalities like recipes, meal planning, calorie tracking, and reminders.
Technique: user interfaces are shown to the supervisor to verify that all necessary features, such as searching for recipes, setting reminders, and monitoring calories. Operate correctly and fulfill customer needs.

5.6 Security Testing

Security testing is essential for the HealthyCooking application because it handles personal and health related data that must be protected from unauthorized access or tampering. Security is a critical non-functional requirement of the system.
Technique: the application prevents users from logging in with incorrect information, ensuring that only authorized users can access the system. Authentication methods, encryption, and the HTTPS protocol are verified to ensure the protection of sensitive data.

5.7 Recovery Testing

Recovery testing verifies that the HealthyCooking application can restore functionality after unexpected failures or network disruptions without losing data or affecting user experience.
Technique: network disruptions or application reboots are simulated to verify that the system automatically reconnects to the server and retrieves information like meal plans and data from backups. The app must resume normal functioning without any loss of data or mistakes.

5.8 Performance Testing

Performance testing confirms the application's capacity to satisfy non-functional criteria like response time, availability, scalability, and stability under user load.
Technique: performance is evaluated with various inputs to confirm responsiveness while exploring recipes, recording meals, or modifying reminders, and to ensure stability and error recovery align with expectations. The system must provide consistent and high performance while fulfilling requirements for response time, availability, and scalability.

5.9 Regression Testing

Regression testing verifies that updates or new functionalities do not harm previously tested features like reminders, browsing recipes, calorie monitoring, and managing profiles.

Technique: following each update, existing components are re-evaluated to confirm proper interactions and to identify any problems arising from recent modifications. This guarantees that the system operates without errors and that all parts keep working correctly, ensuring system stability.

5.10 Acceptance Testing

Acceptance testing confirms that the HealthyCooking app fulfills both functional and non-functional specifications, enabling the customer to approve or ask for changes. Technique: all essential functionalities, including calorie monitoring, recipe exploration, notifications, and user accounts are evaluated based on established acceptance standards, and user input is gathered for needed enhancements. Criteria for Completion: customer approval of the system as a ready, stable, and error free product for release.

5.11 Beta Testing

Beta testing will be performed using a pre-release version of the HealthyCooking application to ensure that the system functions properly in a real-use environment. A selected group of users will try the main features, as recipe browsing, weekly meal planning, calorie tracking, and reminders. Then they will provide feedback on their experience.

The goal of this phase is to identify any faults or usability issues that did not appear during internal testing. All reported defects will be reviewed and corrected to ensure the application is stable, reliable, and ready for final release.

6. PASS / FAIL CRITERIA

Our system must be able to correct all system errors. In this section, we will explain and describe the **Pass/Fail** criteria used in the Healthy Cooking application because we want to minimize failures and improve the system's quality and efficiency. This will facilitate and make it easier to pass all test scenarios, ensuring that functions operate smoothly without errors during runtime. If an error does occur, the system will be able to prevent it.

6.1 Suspension Criteria

Suspension criteria mean that testing activities cannot be resumed or canceled. This includes suspending part or all of the test. In the event of any unforeseen issues, such as the prescription not displaying correctly and missing important information, the test will be temporarily suspended until the problem is resolved within the application. Some of the functions that may cause the test to be suspended include:

- a database connection failure within the application
- inconsistencies or incompatibility of user information with our system
- service outages or malfunctions, technical problems, or system shutdowns

6.2 Resumption Criteria

The old or previous resumption criteria, after their suspension and successful resolution of the problem, will be used for testing. If all tests are passed correctly and successfully, and all specified requirements are met, it will be ready to proceed to the next stage:

- a comprehensive test of the database connection to ensure it is working properly.
- successful implementation of fixes.
- restoring the functionality of dependent functions.

6.3 Approval Criteria

The approval criteria include any error correction measures that may be found during the testing process, resulting in a comprehensive, error-free process that minimizes risks. A test result is only considered passed or acceptable when the specified results meet the required criteria for each test.

7. TESTING PROCESS

This section will identify the methods used in performing test activities. And define the specific techniques for each type of test, and the detailed criteria for evaluating the test results.

7.1 Test Deliverables

The main deliverable document from the test process is this Software Test Plan Document (the STP). After completing the test stage, the project would be ready to deliver.

7.2 Testing Tasks

The Software Test Plan (STP) document should be delivered after the submissions of the Software Requirements (SRS) and the Software Design Specification (SDS) documents. Note that all testing activities and faults should be tracked.

7.3 Responsibilities

All the project team members are responsible of preparing this STP document and for the present and future testing of the system. Also, they are all responsible for all the system errors that may accrue.

7.4 Resources

The needed resources to the test phase include the following in the table below.

Resource	Description
Hardware	Smart Phones
Software	• Internet connection. • iOS and Android operating systems. • Word and documents editing applications.
Members	All group members.

Table 25: Resources information

7.5 Schedule

The following table identifies the schedule for each milestone of the project.

Task	Date
Software Requirement Specification SRS Document	10th of November 2025
Software Design Specification SDS Document	24th of November 2025
Software Test Plan STP Document	6th of December 2025

Table 26: Project milestones submission dates

8. ENVIRONMENTAL REQUIREMENTS

In order to perform the testing process for the HealthyCooking System efficiently and under stable, controlled conditions, a suitable test environment has to be prepared.

This section outlines all the environmental requirements needed in order to perform testing activities: hardware, software, security, tools, publications, and the potential risks and assumptions that may affect the testing phase.

The aim is to ensure that all test procedures run smoothly, the system behaves as expected, and issues, if any, are spotted as early as possible under realistic operating conditions.

8.1 Hardware

The hardware used in the testing environment should be minimal but stable in order for HealthyCooking and its technology to work. The required hardware includes.

• You need a 4 GB RAM or similar smartphone which has good processing power.

• Make sure you have a stable high-speed Wi-Fi or wired internet connection for real-time data retrieval.

• To test the mobile interface, a mobile device, either iOS or Android is needed as it is a mobile-based system.

8.2 Software

The following software components are required to support and execute testing activities.

• For the testing of UI/UX and functionality – iOS and Android Mobile Operating Systems.

• The MySQL Workbench will be used to access and query the HealthyCooking database.

• An IDE like Android Studio or Xcode is a suitable choice for development tools.

• Use Chrome, Safari, or Edge to test the admin dashboard and any web components.

8.3 Security

The test environment should guarantee the protection of user data and not allow unauthorized access in performing the testing activities.

• Secure authentication for testers using authorized accounts only.
• Protection of sensitive data such as calorie logs, user profiles, and meal plans using encryption.
• Ensuring secure access to the database using credentials with limited permissions.

• Safeguarding the sign-in process using Email/Password, OTP, or biometrics (as used in the system).

8.4 Tools

Various tools and methods can be employed in supporting the testing procedures by validating the performance of the system:

• Automated testing tools (optional): Selenium or automated mobile testing frameworks.
• Debugging Tools: IDE-integrated debuggers for code-level verification.
• Database auditing tools: MySQL Enterprise Audit or logs to track query execution and database integrity.
• API testing tools: Postman to validate data exchange and server responses.

8.5 Publications

The following documents are required to guide and support the testing activities:

- Software Requirements Specification (SRS)
- Software Design Specification (SDS)
- User Manual or Application Guide
- Installation instructions (if applicable)
- Project Management Plan (optional)

8.6 Risks and Assumptions

To ensure smooth testing, possible risks and constraints must be identified along with assumptions made during the testing phase.

8.6.1 Test Item Availability

- Some modules or features may not be fully developed or stable at the time of testing.
- Delays in module completion may postpone integration testing.

8.6.2 Test Resource Availability

- Lack of required devices (iOS/Android) may delay UI testing.
- Unavailability of developers or testers may obstruct the testing schedule.

8.6.3 Time Constraints

- Changes in schedule, unexpected delays, or additional fixes may extend the testing timeline.

- If delays occur, contingency actions include adjusting working hours or re-prioritizing test cases.

9. CHANGE MANAGEMENT PROCEDURES

There are methods in place to control and track all modification requests of the Software Test Plan and any project related documents. The method ensures that the requests are properly evaluated, approved, and implemented using an orderly method thereby maintaining system stabilization and eliminating any possibility of any malfunction due to an unexpected issue.

Examples of modification requests that may be required include requirement modifications, test case modifications, test document modifications, system feature/modification configurations, etc. All modification must be documented, executed, and traced using a regulated manner.

The initiation phase of a modification request starts when a developer, tester, supervisor or project manager identifies the requirement of a modification. A formal request for modification must be prepared and submitted detailing the request and affected components. After the submission of the request, the request will be evaluated for validity and effect. The evaluation includes an analysis of the technical, functional and scheduling ramifications of the change request.

The main steps in the change management process include:

- Change Initiation: submission of a change request with justification and scope details.
- Review and Impact Analysis: evaluation by the testing team and project supervisor to assess risks, dependencies, and required updates.
- Approval: authorization by the project manager or QA lead before implementation.
- Implementation: developers apply the approved change following configuration management rules.
- Testing and Validation: affected modules undergo component, integration, or regression testing to ensure system stability.
- Documentation Update: all related documents (STS, test cases, designs, etc.) are revised and assigned new version numbers.
- Closure: the change request is marked complete and stored for traceability.

These procedures ensure that all modifications to the HealthyCooking application and its test documentation are controlled, tracked, and fully validated, supporting organized development and long-term maintainability